HabitLens Project Context

I am building a Flutter application called **HabitLens**. This is intended to become a modern, professional habit tracking application with clean architecture, good UI/UX, statistics, streak tracking, history, reminders, charts, and long-term scalability. The project is written completely in Flutter (Material 3) and is currently tested on a real Android phone using `flutter run`. Every change should compile successfully with `flutter analyze` reporting no issues before it is considered complete.

Current Architecture

The project is organized into folders following a layered structure.

- `models/`
- `habit.dart` – main habit model
- `habit_record.dart` – stores daily completion records
- `app_stats.dart` – statistics model
- `services/`
- `storage_service.dart`
- `streak_service.dart`
- `stats_service.dart`
- `providers/`
- `theme_provider.dart`
- `habit_provider.dart` (planned to become the central state manager)
- `repositories/`
- `habit_repository.dart` (planned for data abstraction)
- `screens/`
- `home_screen.dart`
- `main_screen.dart`
- `stats_screen.dart`
- `settings_screen.dart`
- `edit_habit_screen.dart`
- `habit_detail_screen.dart` (planned)
- `widgets/`
- `habit_tile.dart`

- `stat_card.dart`
- `completion_chart.dart`
- `empty_state.dart`
- `utils/`
  - date utilities

The project is also stored in a private GitHub repository and uses Git for version control.

Storage

The application currently stores all data locally using `SharedPreferences`.

Storage currently saves:

- Habits
- Daily completion records
- Streak values
- Total completions
- Longest streak
- Current streak
- Dark mode preference

Data is stored as JSON by converting models using `toJson()` and `fromJson()`.

The architecture should make it possible to replace SharedPreferences with Hive, Isar, SQLite or Firebase later without changing the UI.

Habit Model

Each habit currently contains:

- id
- name
- createdDate
- currentStreak
- longestStreak
- totalCompletions
- archived flag
- list of HabitRecord objects

Each HabitRecord stores:

- date (YYYY-MM-DD)
- completed (bool)

Currently completion is tracked once per day.

Current Features

Already implemented:

- Add habit
- Edit habit name
- Delete selected habits
- Multi-selection by long press
- Daily checkbox
- Daily completion persistence
- Automatic streak calculation
- Longest streak calculation
- Total completion count
- Statistics calculation
- Dark mode
- Bottom navigation
- Statistics screen
- Settings screen
- Local persistence
- Material 3 UI
- GitHub repository
- Git version control

Statistics

Current statistics include:

- Total habits
- Total completions
- Longest streak
- Completion percentage

Statistics are calculated by StatsService.

Streak Logic

Completion is recorded per day.

When today's checkbox changes:

- today's HabitRecord is created or updated
- streaks are recalculated
- total completions are recalculated
- longest streak updates automatically
- data is saved immediately

Theme

Theme uses Provider.

Dark mode preference is stored in SharedPreferences.

The final design should expose theme switching only inside Settings.

The AppBar should not contain a separate dark mode icon.

Home Screen

Current functionality:

- Shows all habits
- Checkbox for today's completion
- Current streak
- Total completions
- Long press enters selection mode
- Edit selected habit
- Delete selected habits
- Floating Action Button adds new habit

Planned improvements:

Completed habits should automatically move below incomplete habits.

Example:

TO DO

☐ Workout

☐ Reading

Completed Today

☑ Meditation

☑ Water

A separator should only appear if both sections contain items.

The AppBar should remove:

- statistics icon
- dark mode icon

because both features already exist through bottom navigation and Settings.

Bottom Navigation

Current tabs:

- Habits
- Statistics
- Settings

Planned Habit Detail Screen

Tapping a habit should open a detailed page showing:

- Habit name
- Current streak
- Longest streak
- Total completions
- Completion percentage
- Today's status
- Monthly calendar
- Recent activity
- Charts
- Notes
- Reminder settings
- Edit
- Archive
- Delete

Planned Calendar

Each habit should have a monthly calendar.

Color coding:

Green = completed

Red = missed

Gray = future

The calendar becomes the main history view.

Planned Charts

The project already contains a completion_chart widget but charts are not yet fully implemented.

Future charts:

- Weekly completion
- Monthly completion

- Yearly completion
- GitHub-style contribution heatmap
- Streak graph
- Completion trend graph

Planned Habit Groups

Future versions should support grouping habits.

Examples:

Morning

Fitness

Study

Health

Work

Personal

Each group should be collapsible.

Planned Reminders

Notifications should support:

- daily reminder
- custom reminder time
- multiple reminders
- enable/disable per habit

Planned Habit Editing

Editing should eventually allow:

- name
- icon
- color
- group
- reminder
- archive
- delete

Planned Statistics

Future statistics:

- average streak
- completion heatmap
- best day
- worst day
- monthly consistency
- yearly consistency
- completion trends
- missed days
- completion percentage by habit
- total active days
- longest inactive period

Planned Features

The long-term roadmap includes:

- Calendar
- Charts
- Heatmaps
- Habit notes
- Archive
- Search
- Sorting
- Filters
- Categories
- Icons
- Colors
- Backup
- Restore
- Import/export
- CSV export
- Google Drive backup
- Local backup
- Firebase sync (optional)
- Animations
- Swipe actions
- Achievements
- Badges
- Milestones
- Reminder notifications
- Widgets
- Wear OS support (possible future)

Code Quality Requirements

Every new feature should:

- compile successfully
- pass flutter analyze
- preserve existing functionality
- integrate into the existing architecture
- avoid duplicated code
- keep UI clean
- remain scalable

Avoid unnecessary rewrites of working code. Prefer extending the current architecture instead of replacing it.

The goal is to turn HabitLens into a polished, production-quality Flutter habit tracker with clean architecture, excellent UX, extensibility, and maintainable code.